Artificial Intelligence

Problem Solving by Informed Search

Conducted by:

Dr. Fazlul Hasan Siddiqui

(fazlul.siddiqui@duet.ac.bd)

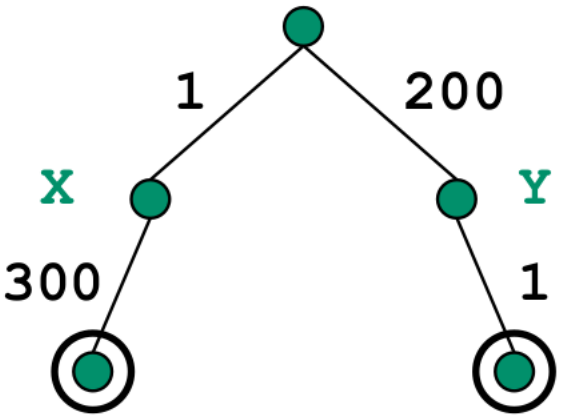
Credit: MIT, USA

Uninformed vs. Informed Search

- Depth-first, breadth-first and uniform-cost searches are uninformed.
- In informed search there is an estimate available of the cost (distance) from each state (city) to the goal.
- This estimate (heuristic) can help you head in the right direction.
- Heuristic embodied in function h(n), estimate of remaining cost from search node n to the least cost goal.
- Graph being searched is a graph of states. Search algorithm defines a tree of search nodes. Two paths to the same state generate two different search nodes.
- Heuristic could be defined on underlying state; the path to a state does not affect estimate of distance to the goal.

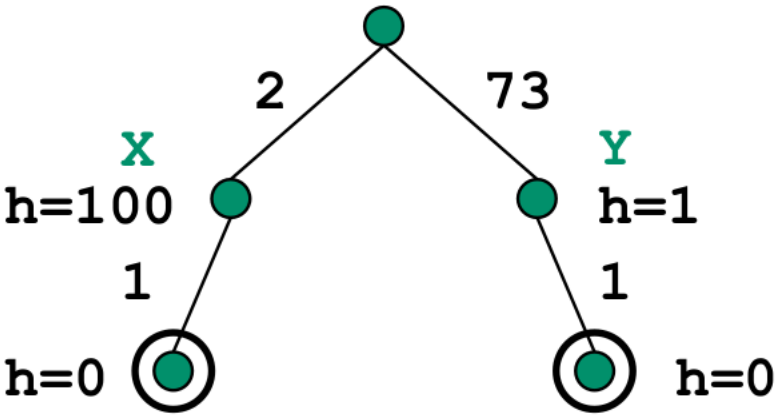
Using Heuristic Information

- Should we go to X or Y?
- Uniform cost says go to X
- If $h(X) \gg h(Y)$, this should affect our choice
- If g(n) is path-length of node n, we can use g(n) + h(n) to prioritize the agenda
- This method is called A^* [pronounced "A star"]



Admissibility

- What must be true about h for A^* to find optimal path?
- A^* finds optimal path if h is admissible; h is admissible when it never overestimates.
- In this example, h is not admissible.
- In route finding problems, straight-line distance to goal is admissible heuristic.

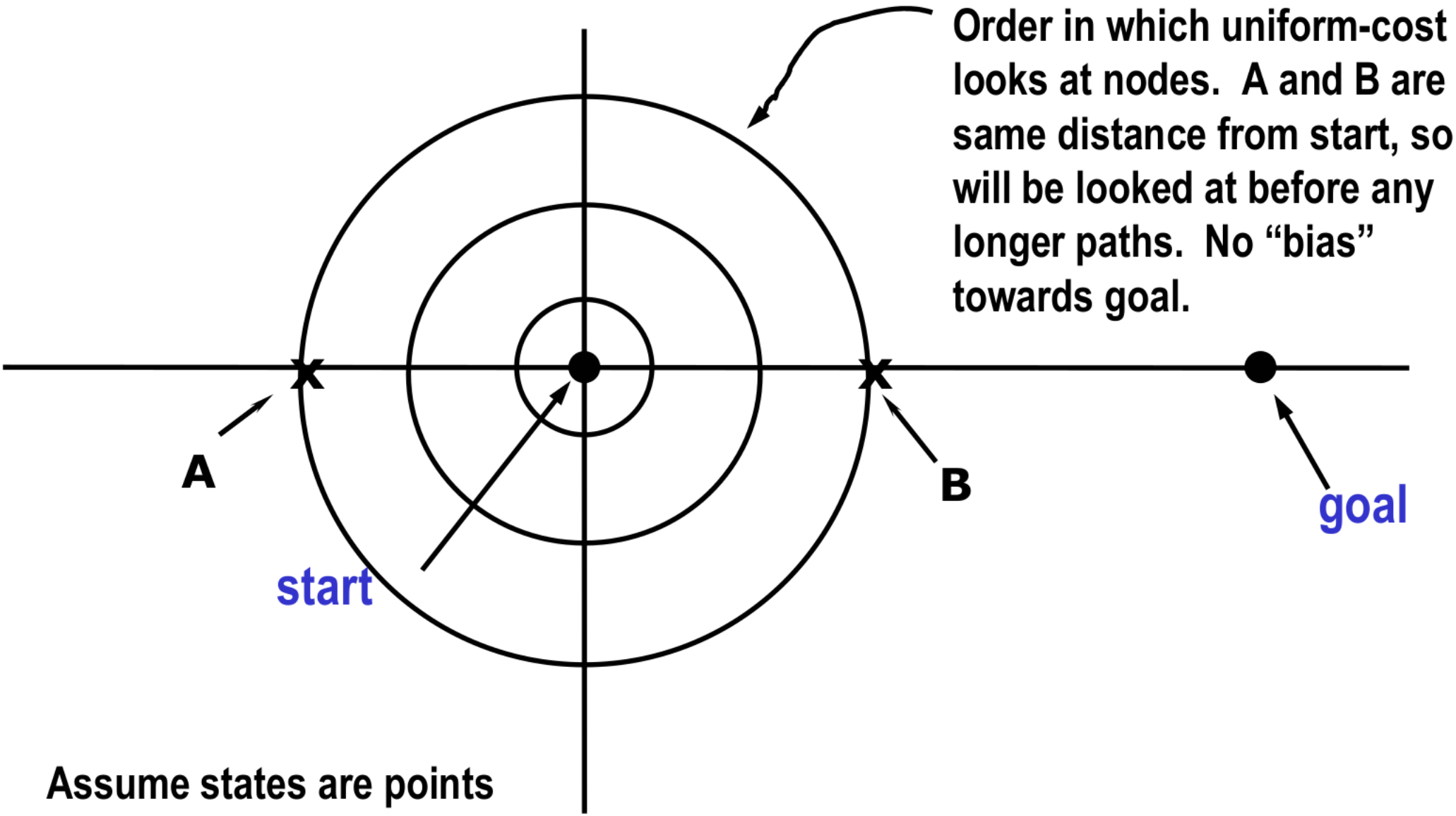


g(X)+h(X) = 102

g(Y)+h(Y) = 74

Optimal path is not found!

Why use estimate of goal distance?



Assume states are points the Euclidean plane.

A* search: Minimizing the total estimated solution cost

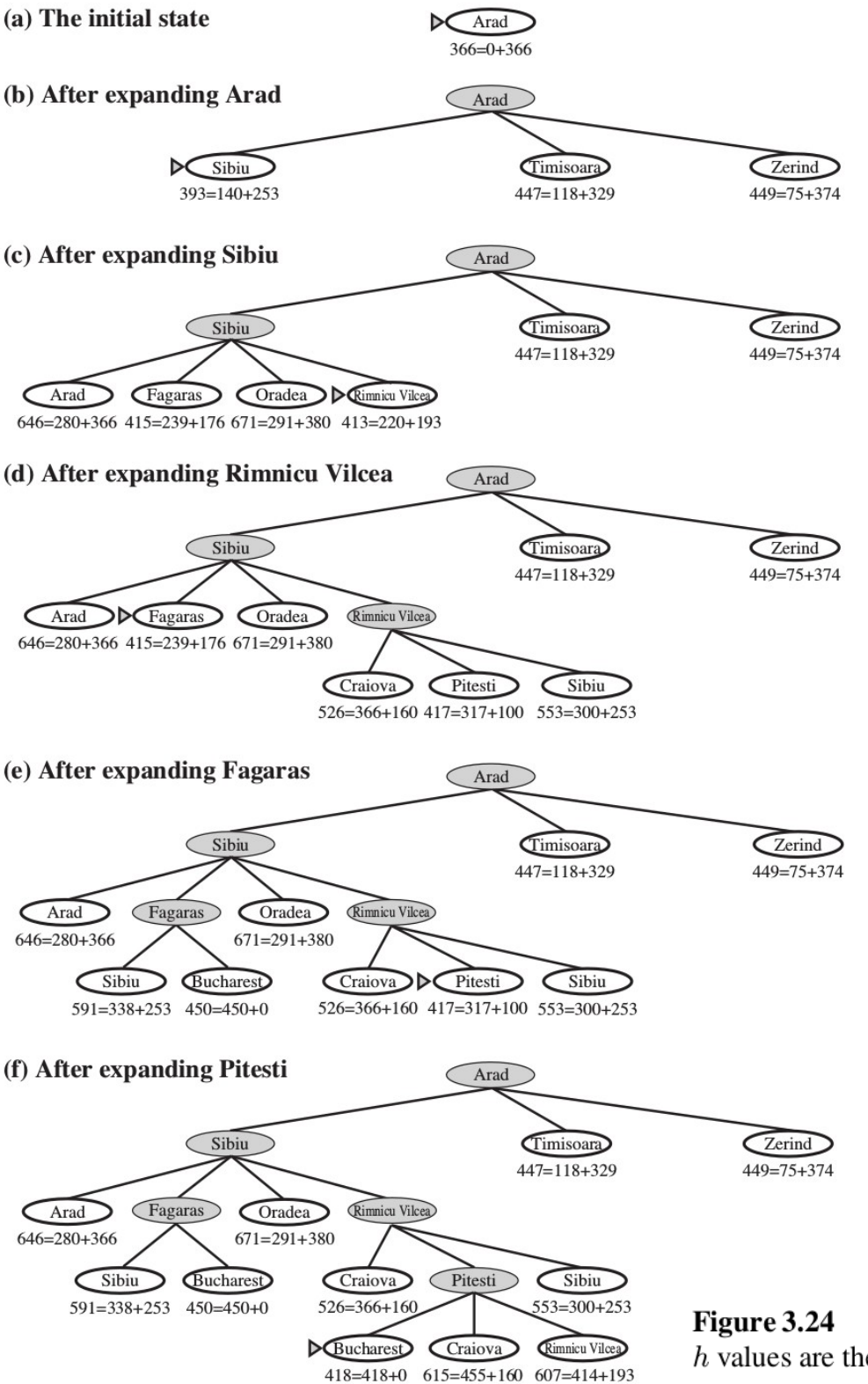
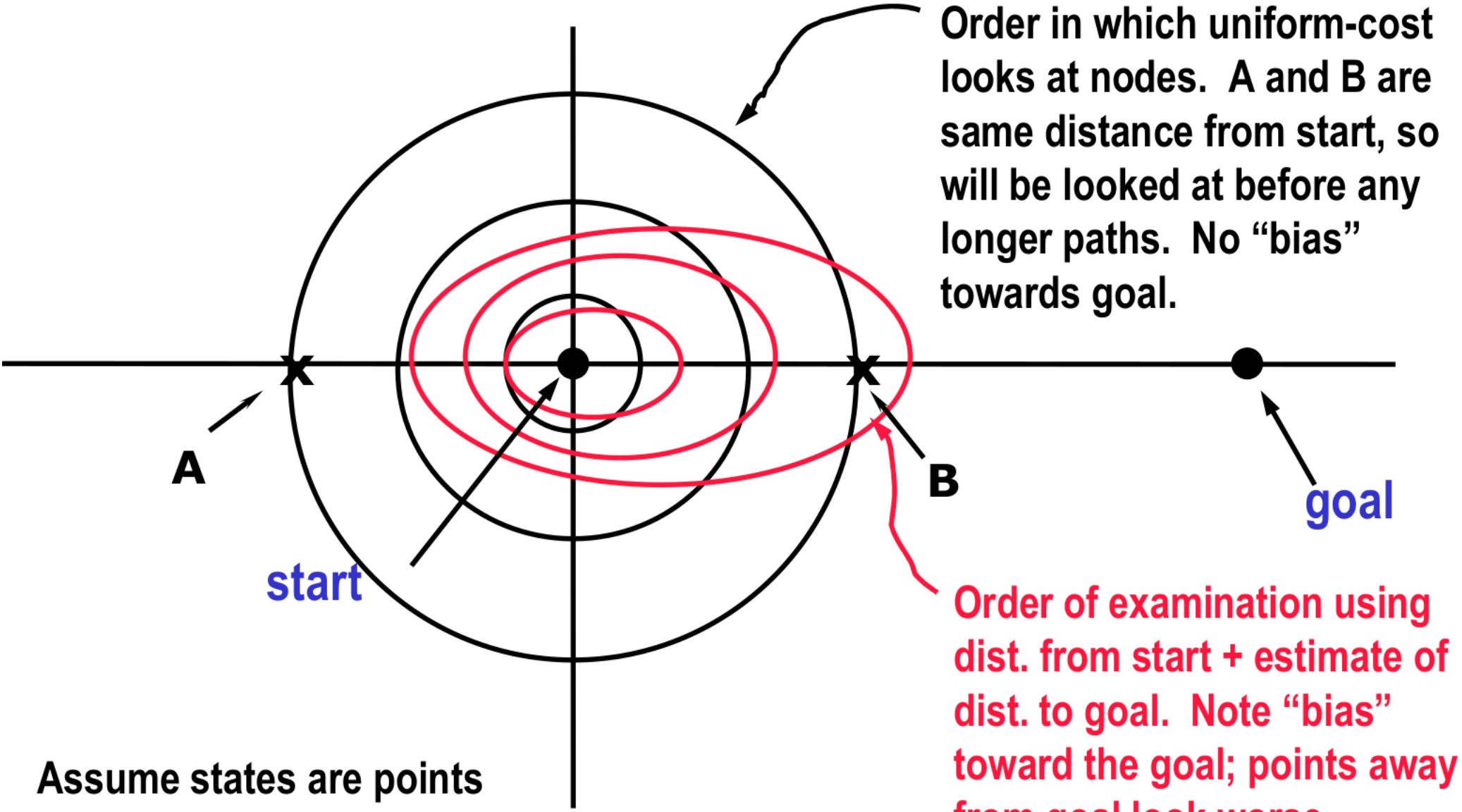


Figure 3.24 Stages in an A* search for Bucharest. Nodes are labeled with $f = g + h$. The h values are the straight-line distances to Bucharest taken from Figure 3.22.

Why use estimate of goal distance?



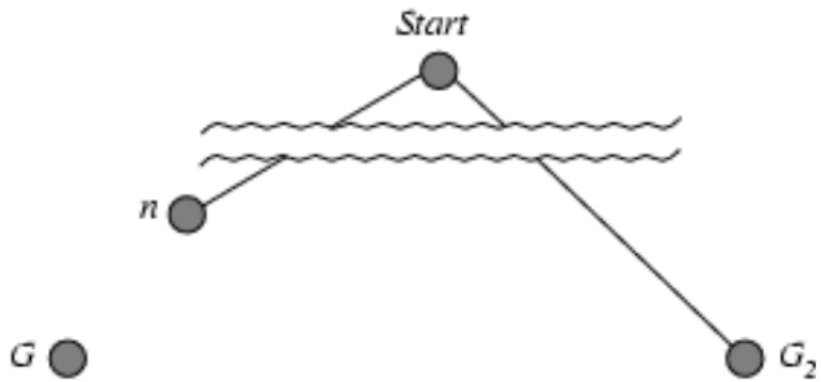
Assume states are points the Euclidean plane.

Admissible heuristics

- A heuristic $h(n)$ is admissible if for every node n, $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal state from n.
- An admissible heuristic never overestimates the cost to reach the goal, i.e., it is optimistic
- Example: $h_{SLD}(n)$ (never overestimates the actual road distance)
- Theorem: If $h(n)$ is admissible, A* using TREE-SEARCH is optimal

Optimality of A* (proof)

- Suppose some suboptimal goal G_2 has been generated and is in the fringe. Let *n* be an unexpanded node in the fringe such that *n* is on a shortest path to an optimal goal *G*.



- $f(G_2) = g(G_2)$ since $h(G_2) = 0$
- $g(G_2) > g(G)$ since G_2 is suboptimal
- $f(G) = g(G)$ since $h(G) = 0$
- $f(G_2) > f(G)$ from above
- $h(n) \leq h^*(n)$ since h is admissible
- $g(n) + h(n) \leq g(n) + h^*(n)$
- $f(n) \leq f(G)$
- Hence f(G2) > f(n), and A* will never select G2 for expansion

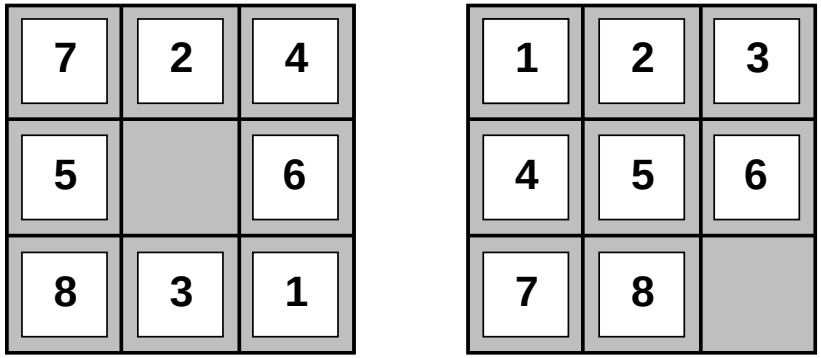
Heuristics

- If we set h=0, then A^* is uniform-cost search; h=0 is admissible heuristic (when all costs are non-negative).
- Very difficult to find heuristics that guarantee sub-exponential worst-case cost.
- Heuristic functions can be solutions to "relaxed" version of original problem, e.g. straight line distance is solution to route-finding when we relax constraint to follow the roads.

Admissible heuristics

E.g., for the 8-puzzle:

$h_1(n)$ = number of misplaced tiles
$h_2(n)$ = total Manhattan distance
(i.e., no. of squares from desired location of each tile)



Start State **Goal State**

$h_1(S) =$?? 6
$h_2(S) =$?? 4+0+3+3+1+0+2+1 = 14

Dominance

Given two admissible heuristics h_a, h_b,
if $h_b(n) \geq h_a(n)$ for all n then h_b dominates h_a and is better for search

In the 8-puzzle h_2 dominates h_1. Typical search costs:

$d = 14$ IDS = 3,473,941 nodes
A$^*(h_1)$ = 539 nodes
A$^*(h_2)$ = 113 nodes
$d = 24$ IDS $\approx$ 54,000,000,000 nodes
A$^*(h_1)$ = 39,135 nodes
A$^*(h_2)$ = 1,641 nodes

Given two admissible heuristics h_a, h_b,

$$h(n) = \max(h_a(n), h_b(n))$$

is also admissible and dominates h_a, h_b

Relaxed problems

Admissible heuristics can be derived from the **optimal** solution cost of a **relaxed** version of the problem

Rules of the 8-puzzle:

a tile can move from square A to square B if
A is adjacent to B **and** B is blank;

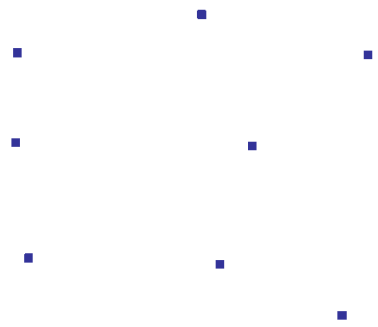
Relaxations:

- ♢ a tile can move from square A to square B if A is adjacent to B
- ♢ a tile can move from square A to square B if B is blank
- ♢ a tile can move from square A to square B

Key point: the optimal solution cost of a relaxed problem is no greater than the optimal solution cost of the real problem

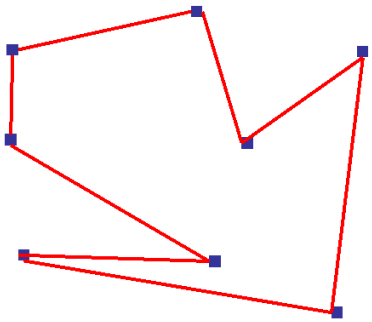
Relaxed problems contd.

Well-known example: travelling salesperson problem (TSP)
Find the shortest tour visiting all cities exactly once and returning to first city



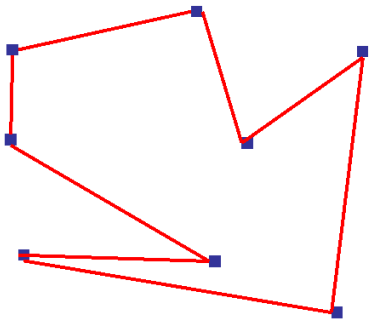
Relaxed problems contd.

Well-known example: travelling salesperson problem (TSP)
Find the shortest tour which visits all cities exactly once and then returns to first city



Relaxed problems contd.

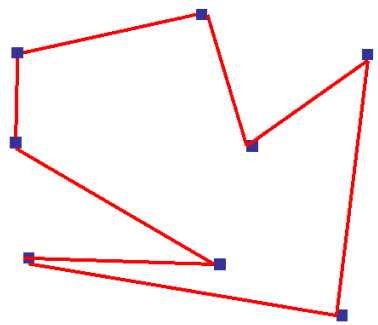
Well-known example: travelling salesperson problem (TSP)
Find the shortest tour which visits all cities exactly once and then returns to first city



The set of arcs which covers all nodes at minimum cost
and forms a tour

Relaxed problems contd.

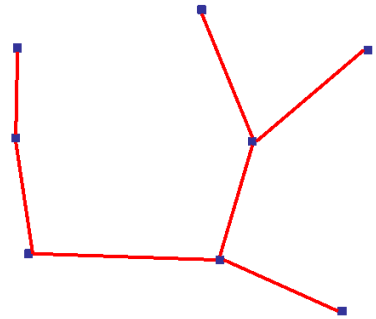
Well-known example: travelling salesperson problem (TSP)
Find the shortest tour which visits all cities exactly once and then returns to first city



The set of arcs which covers all nodes at minimum cost
so that each node has degree 2

Relaxed problems contd.

Minimum spanning tree (MST)
The set of arcs which covers all nodes at minimum cost



Minimum spanning tree can be computed in $O(n^2)$
and is a lower bound on the shortest (open) tour

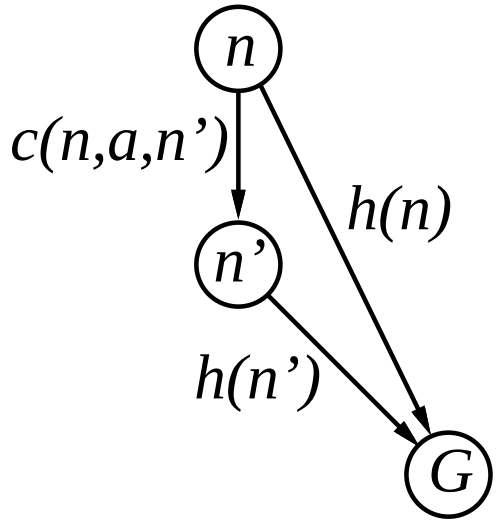
Consistency

A heuristic is consistent if

$$h(n) \leq c(n, a, n') + h(n')$$

If h is consistent, then h is admissible, and $f(n)$ is nondecreasing along any path:

$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= g(n) + c(n, a, n') + h(n') \\ &\geq g(n) + h(n) \\ &= f(n) \end{aligned}$$

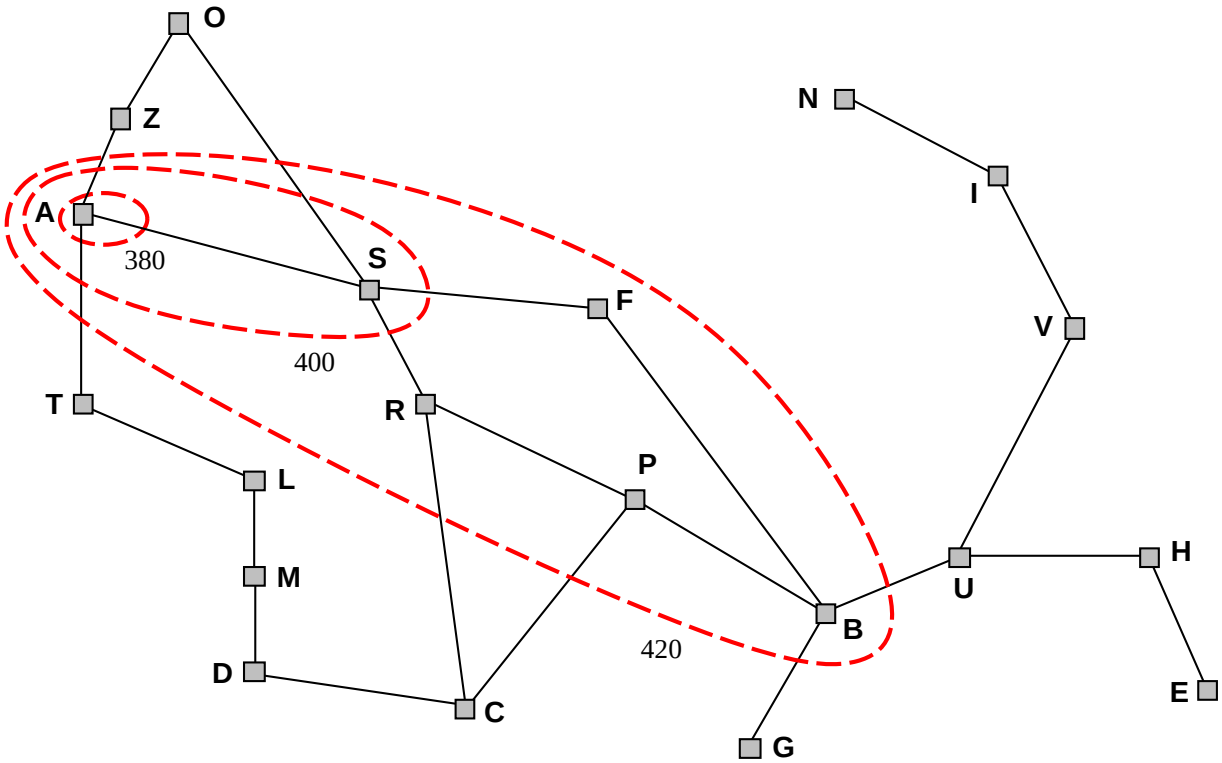


Consequently, when expanding a node, we cannot get a node with a smaller f, and so the value of the best node on the fringe will never decrease.

Optimality of A* (based on consistency)

Consistency: A* expands nodes in order of increasing f value

Gradually expands "f-contours" of nodes (cf. breadth-first expands layers, uniform-cost expands g-contours)
Contour i has all nodes with $f = f_i$, where $f_i < f_{i+1}$



Properties of A*

Complete?? Yes, unless there are infinitely many nodes with $f \leq f(G)$

Time?? Exponential in [relative error in h $\times$ length of soln.]

Space?? Keeps all nodes in memory

Optimal?? Yes—cannot expand f_{i+1} until f_i is finished

A* expands all nodes with $f(n) < C^*$
A* expands some nodes with $f(n) = C^*$
A* expands no nodes with $f(n) > C^*$

(Heuristic) Iterative Deepening: IDA*

- Like Iterative Deepening DFS
  - But the depth bound is measured in terms of the f value
- If you don't find a solution at a given depth
  - Increase the depth bound:
    to the minimum of the f-values that exceeded the previous bound

Best-first search

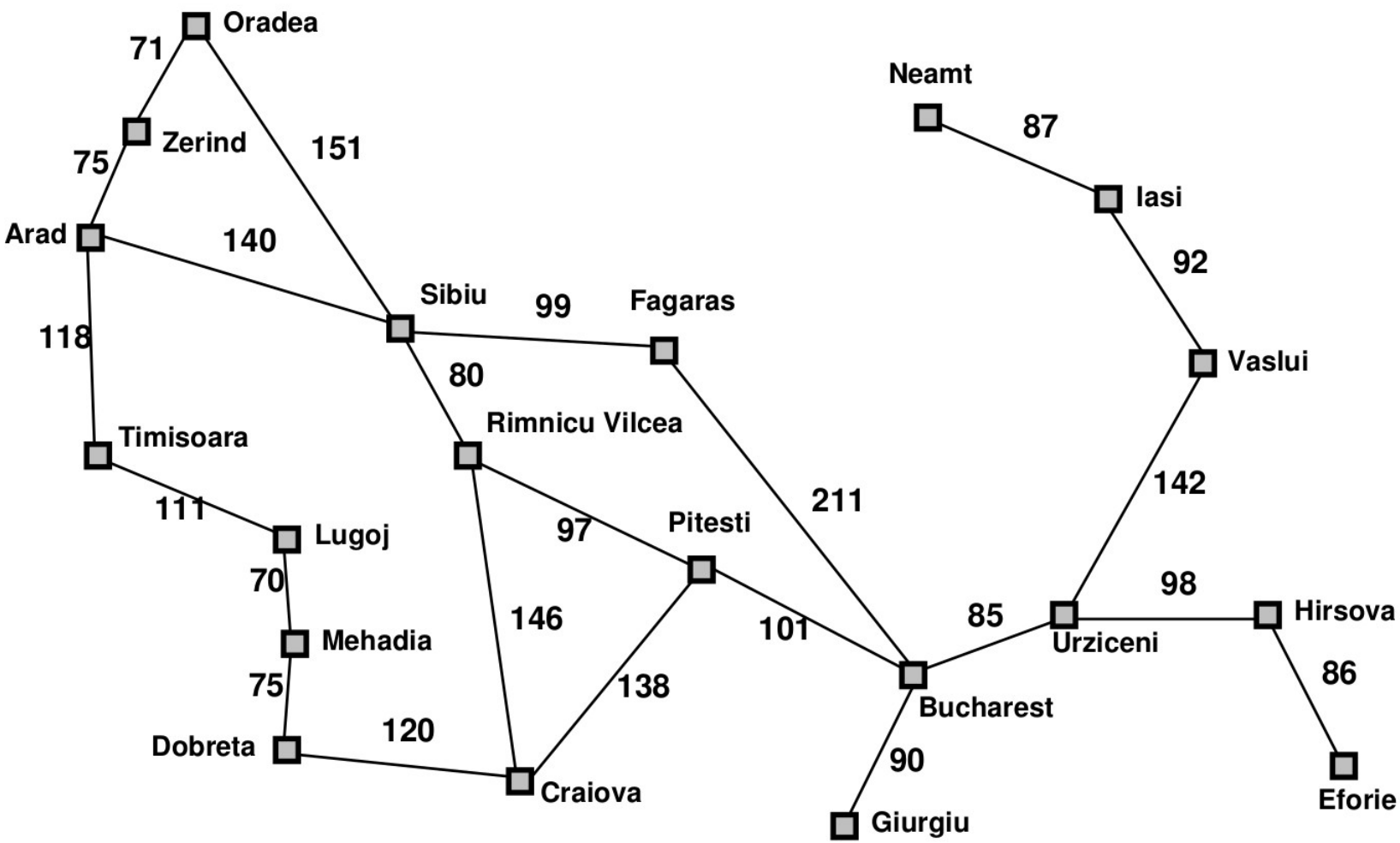
- The general approach we will consider is called **best-first search.** Best-first search is an instance of the general TREE-SEARCH or GRAPH-SEARCH algorithm in which a node is selected for expansion based on an **evaluation function,** f (n) .

- <u>Implementation</u>:
  - Best-first search can be implemented within our general search framework via a priority queue, a data structure that will maintain the fringe in ascending order of f –values.
  - A key component of these algorithms is a heuristic function, denoted by h(n):

    h(n) = estimated cost of the cheapest path from node n to a goal node.

    if *n* is a goal node, then *h(n)* = 0.

Romania with step costs in km

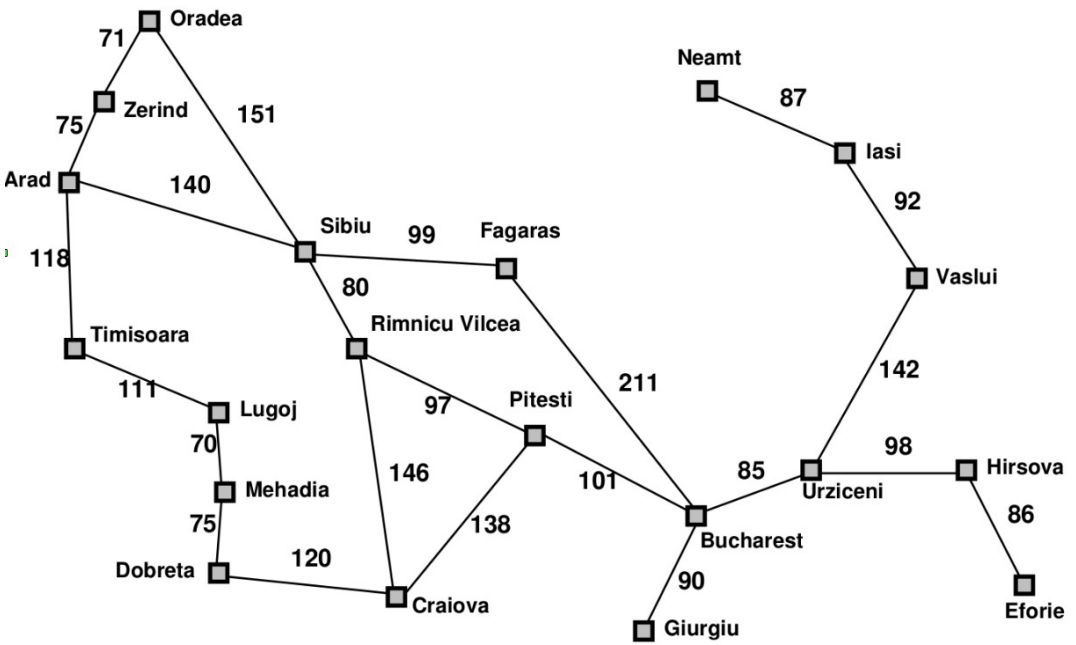


Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy best-first search

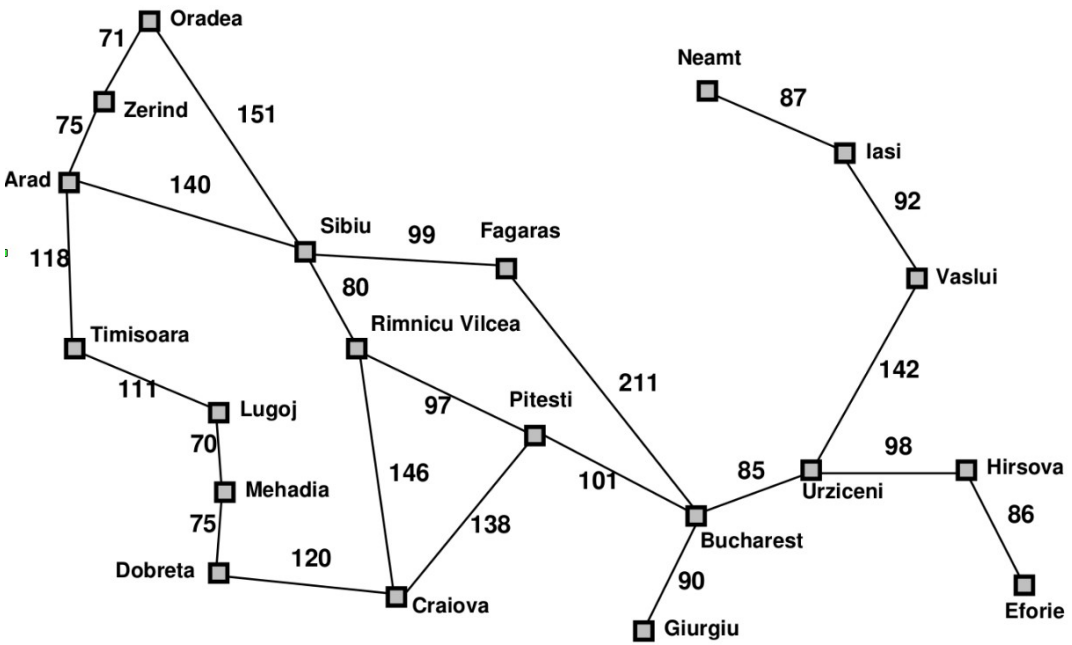
- Greedy best-first search tries to expand the node that is closest to the goal.
- Evaluation function $f(n) = h(n)$ (heuristic)
- e.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest
- Greedy best-first search expands the node that appears to be closest to goal

Greedy Best-first search

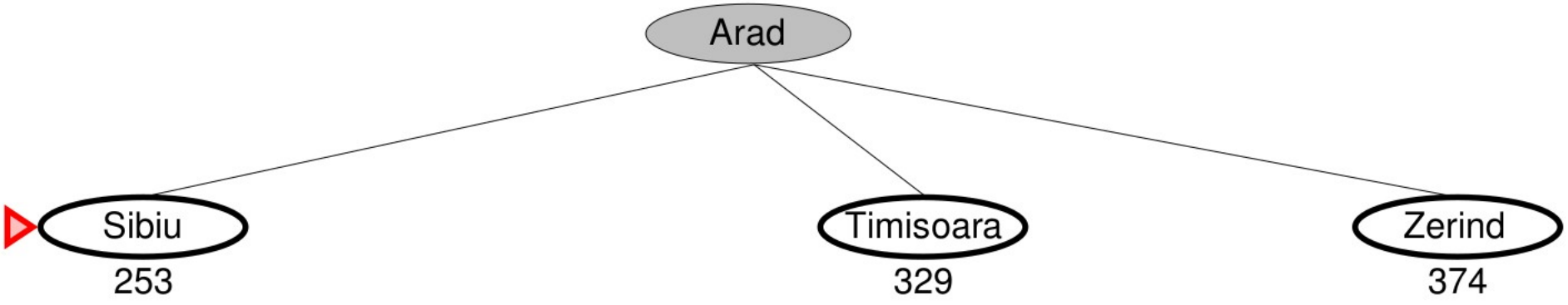


Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

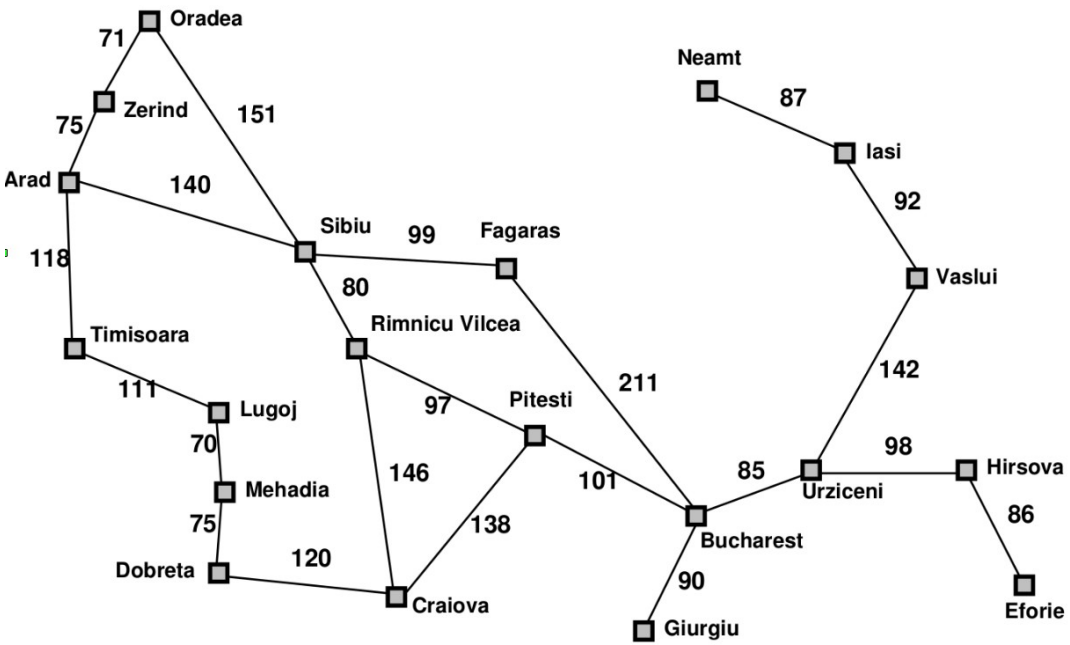
Greedy Best-first search



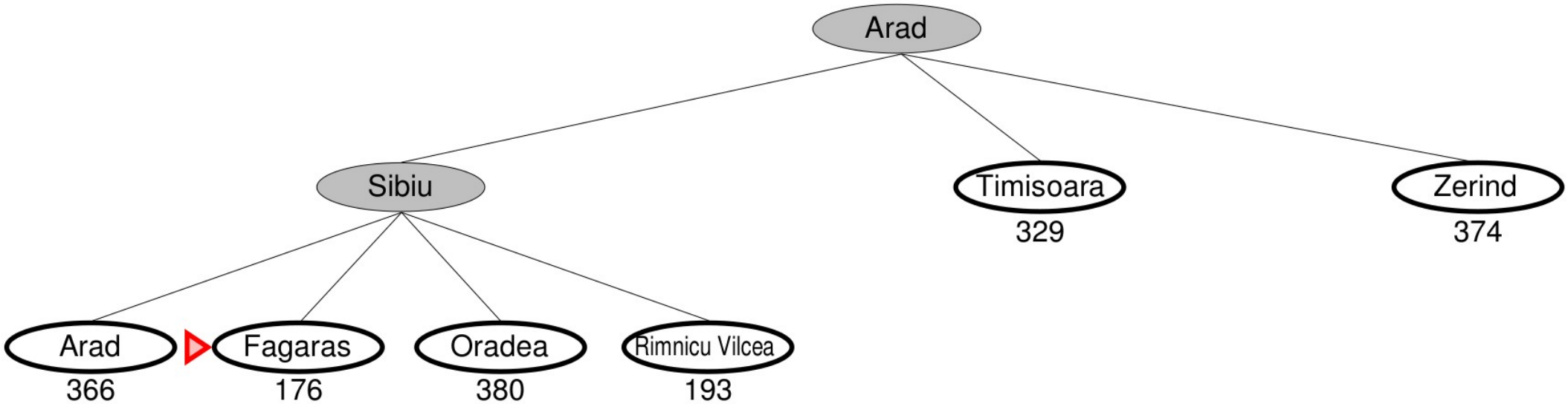
Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



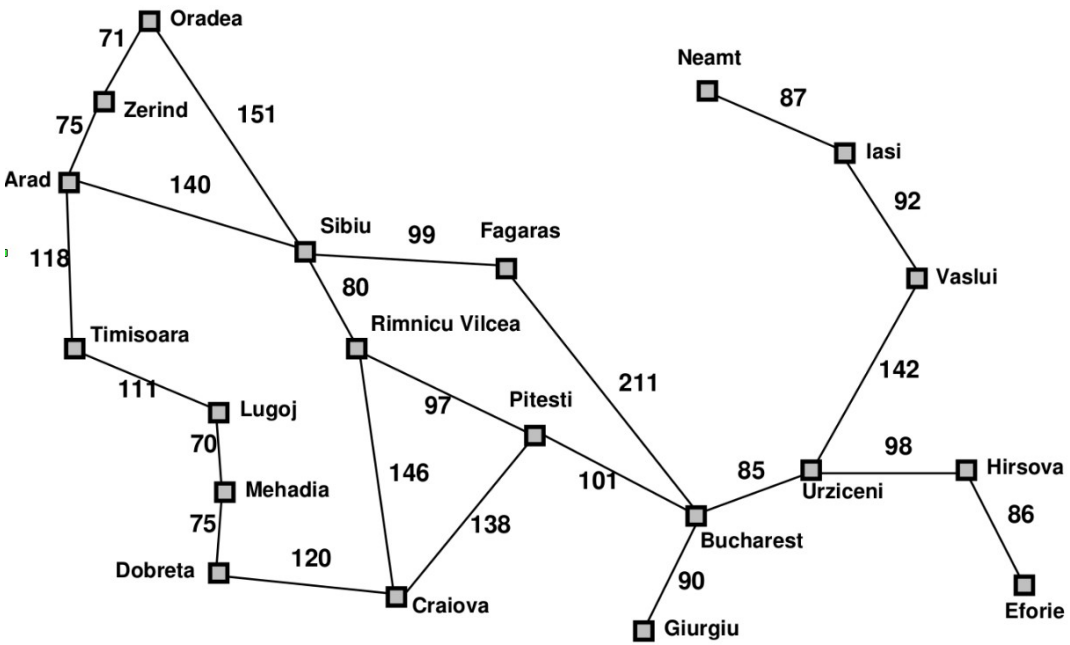
Greedy Best-first search



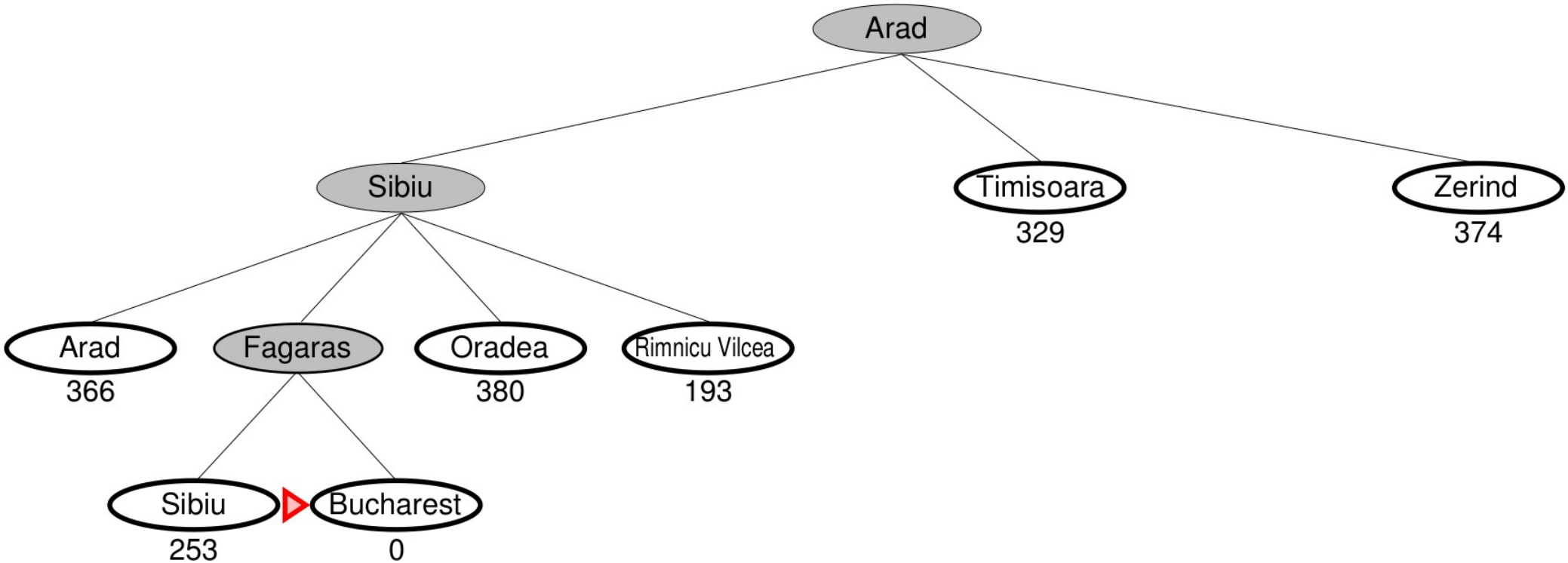
Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



Greedy Best-first search



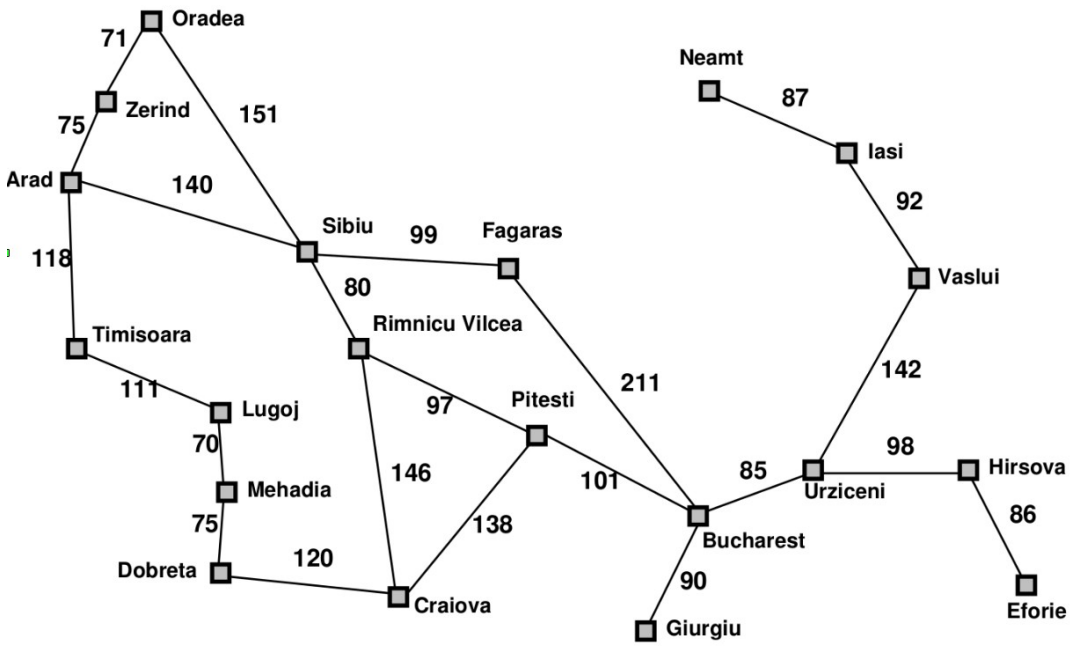
Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



Recursive best-first search (RBFS)

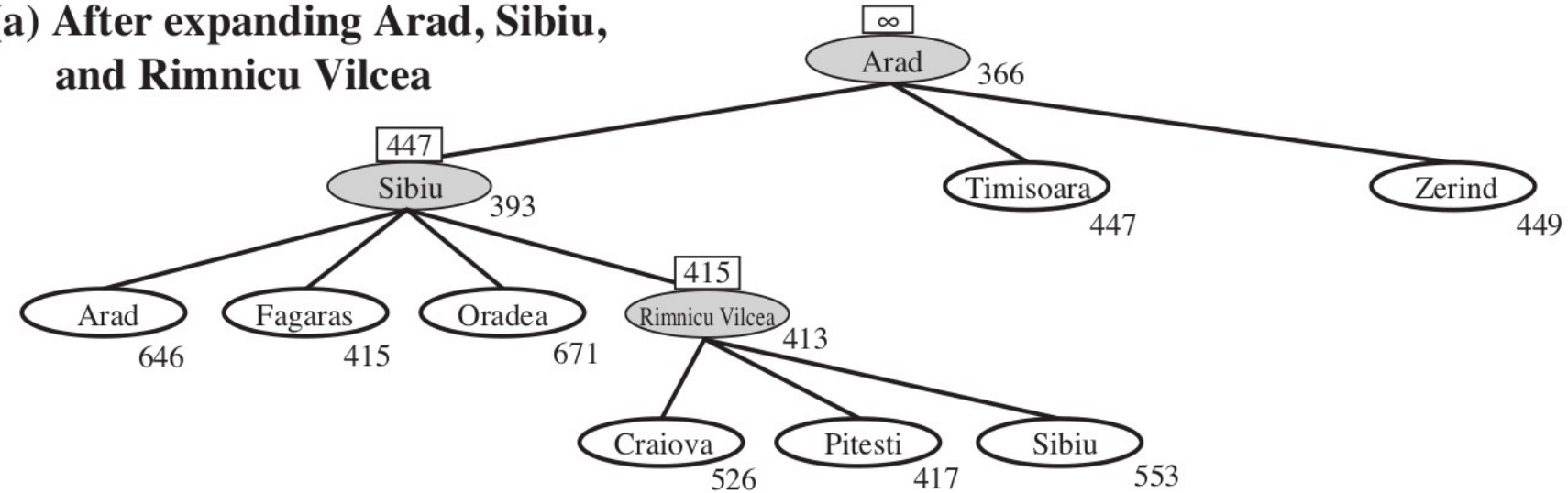
- (RBFS) is **a** simple recursive algorithm that attempts to mimic the operation of standard best-first search, but using only linear space.
- Its structure is similar to that of a recursive depth-first search, but rather than continuing indefinitely down the current path it keeps track of the f-value of the best alternative path available from any ancestor of the current node.
- If the current node exceeds this limit, the recursion unwinds back to the alternative path.
- As the recursion unwinds, RBFS replaces the *f* -value of each node along the path with the best *f* -value of its children.

Recursive best-first search (RBFS)

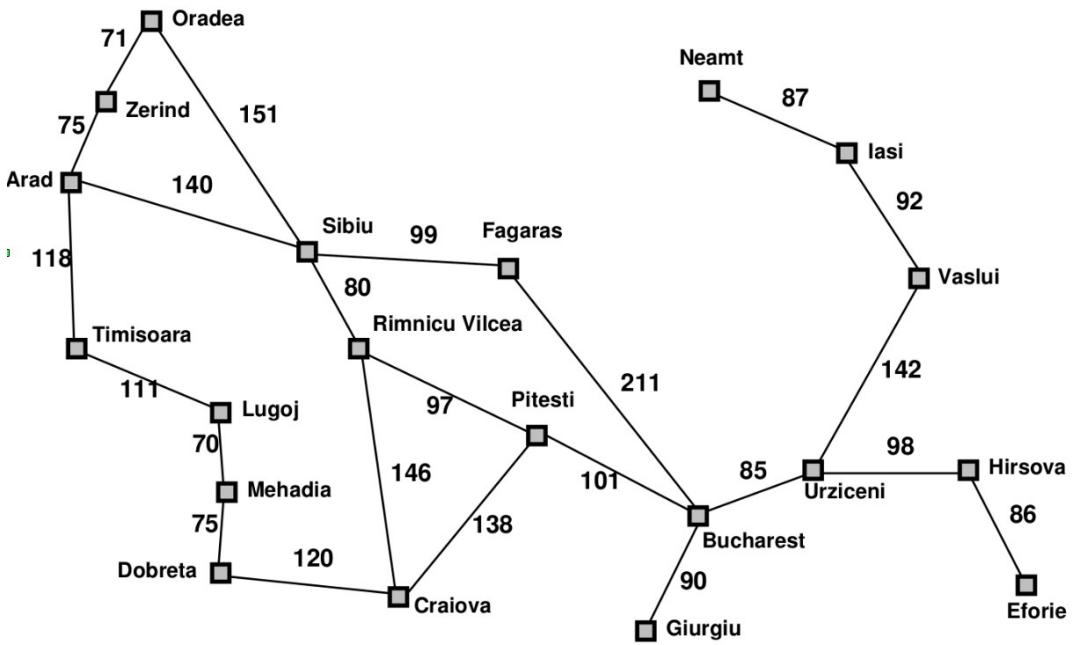


Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

(a) After expanding Arad, Sibiu, and Rimnicu Vilcea



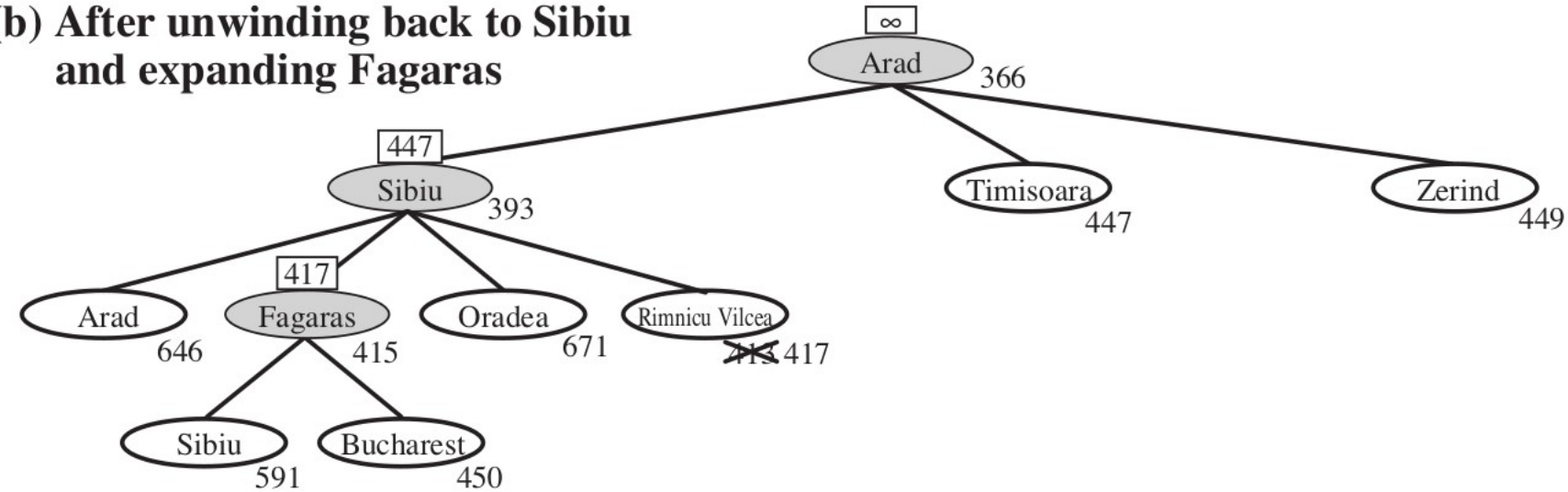
Recursive best-first search (RBFS)



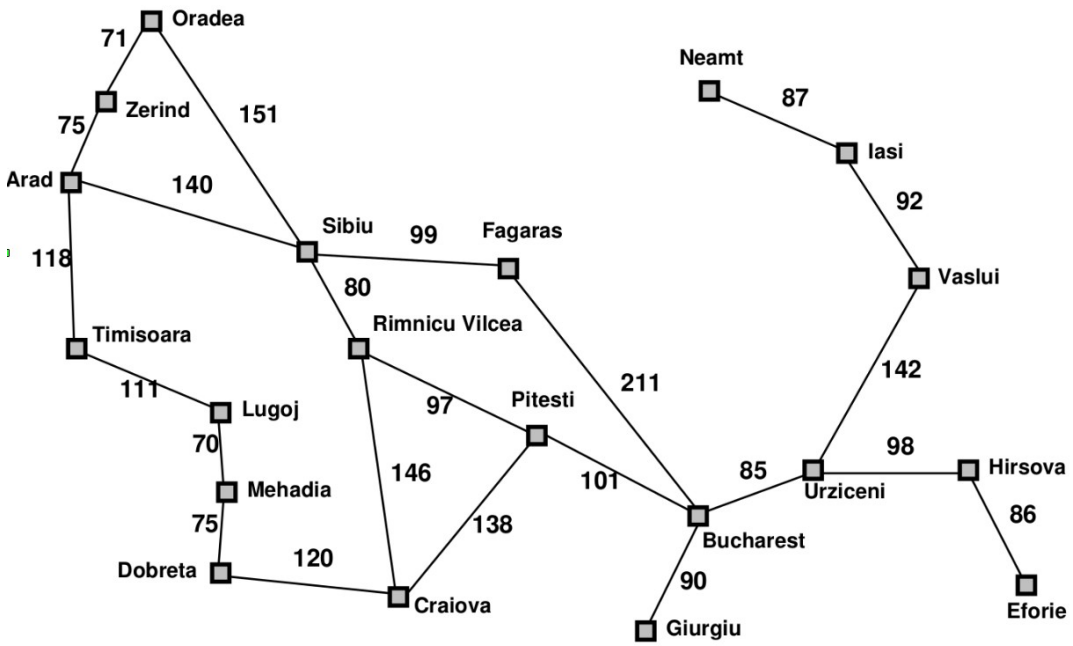
Straight–line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

(b) After unwinding back to Sibiu and expanding Fagaras

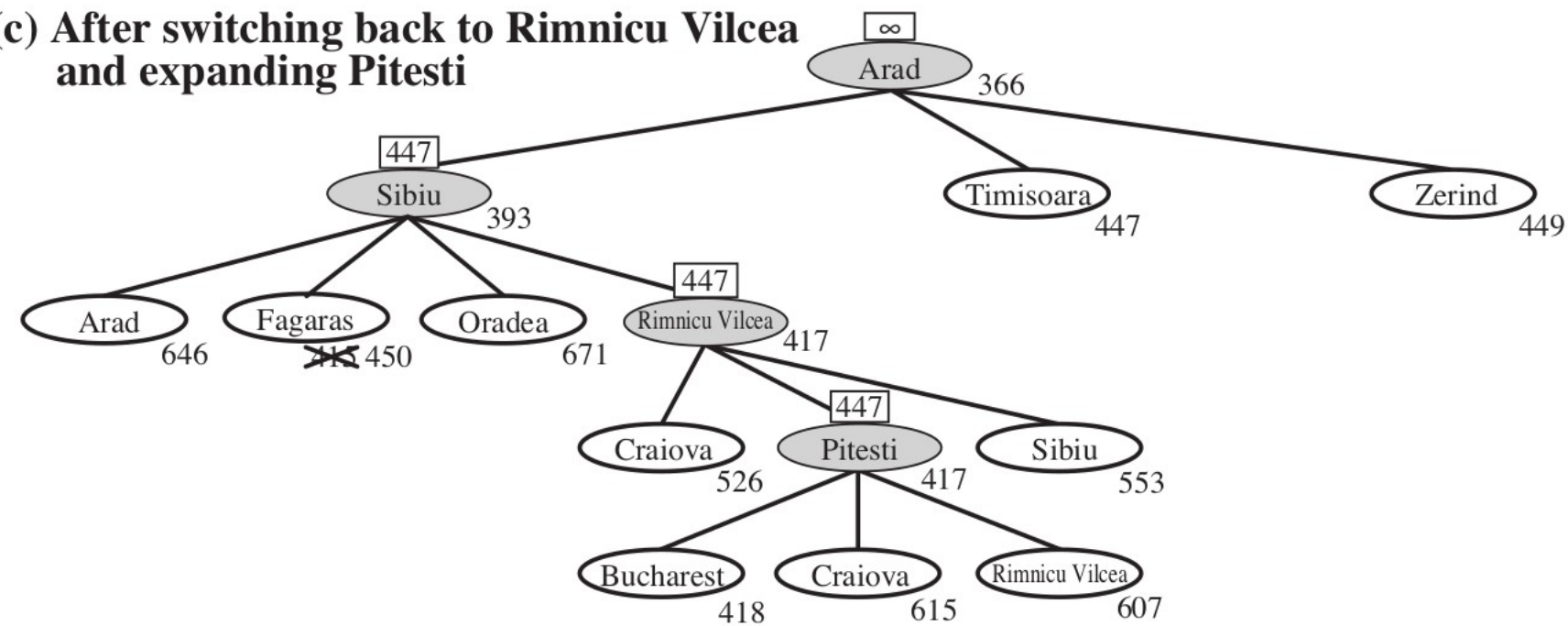


Recursive best-first search (RBFS)



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

(c) After switching back to Rimnicu Vilcea and expanding Pitesti

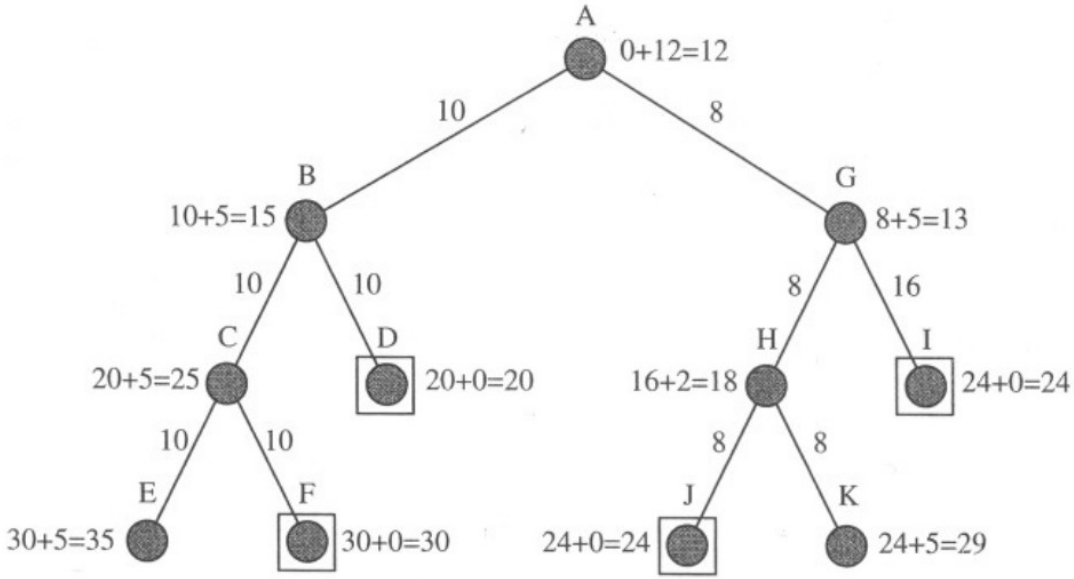


Recursive best-first search (RBFS)

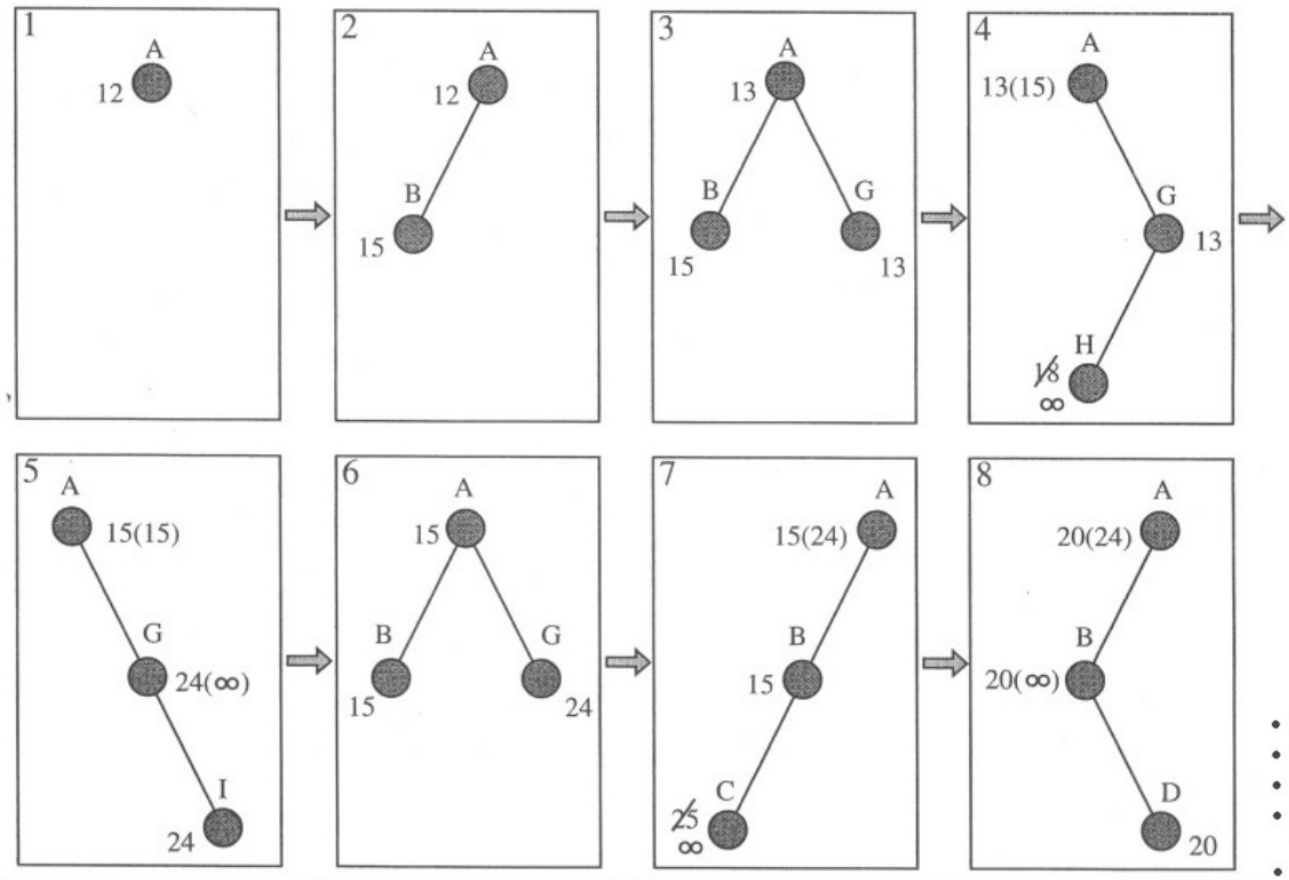
- RBFS is somewhat more efficient than IDA*, but still suffers from excessive node re-generation.
  - RBFS follows the path via Rimnicu Vilcea, then “changes its mind” and tries Fagaras, and then changes its mind back again.
  - These mind changes occur because every time the current best path is extended, its f -value is likely to increase—h is usually less optimistic for nodes closer to the goal.
  - When this happens, the second-best path might become the best path, so the search has to backtrack to follow it.
- Like A* tree search, **RBFS is an optimal algorithm if the heuristic function h(n) is admissible.**
- RBFS's **space complexity is linear in the depth of the deepest optimal solution**, but its time complexity is rather difficult to characterize: it depends both on the accuracy of the heuristic function and on how often the best path changes as nodes are expanded.

Drawbacks of RBFS & IDA*

- IDA* and RBFS suffer from using too little memory.
- Between iterations, IDA* retains only a single number: the current f -cost limit. RBFS retains more information in memory, but it uses only linear space: even if more memory were available, RBFS has no way to make use of it.
- Because they forget most of what they have done, both algorithms may end up reexpanding the same states many times over.
- Two algorithms that do this are MA* (memory-bounded A*) and SMA* (simplified MA*).



1. At each stage, one successor is added to the deepest lowest-f-cost node that has some successors not currently in the tree. The left child B is added to the root A.
2. Now $f(A)$ is still 12, so we add the right child G (f = 13). Now that we have seen all the children of A, we can update its f-cost to the minimum of its children, that is, 13. The memory is now full.
3. G is now designated for expansion, but we must first drop a node to make room. We drop the shallowest highest-f-cost leaf, that is, B. When we have done this, we note that A's best forgotten descendant has f = 15, as shown in parentheses. We then add H, with $f(H)$ = 18. Unfortunately, H is not a goal node, but the path to H uses up all the available memory. Hence, there is no way to find a solution through H, so we set $f(H) = \infty$.
4. G is expanded again. We drop H, and add I, with $f(I)$ = 24. Now we have seen both successors of G, with values of ∞ and 24, so $f(G)$ becomes 24. $f(A)$ becomes 15, the minimum of 15 (forgotten successor value) and 24. Notice that I is a goal node, but it might not be the best solution because A's f-cost is only 15.
5. A is once again the most promising node, so B is generated for the second time. We have found that the path through G was not so great after all.
6. C, the first successor of B, is a nongoal node at the maximum depth, so $f(C) = \infty$.
7. To look at the second successor, D, we first drop C. Then $f(D)$ = 20, and this value is inherited by B and A.
8. Now the deepest, lowest-f-cost node is D. D is therefore selected, and because it is a goal node, the search terminates.



SMA* search

- It will utilize whatever memory is made available to it.
- It avoids repeated states as far as its memory allows.
- It is complete if the available memory is sufficient to store the *shallowest* solution path.
- It is optimal if enough memory is available to store the shallowest optimal solution path. Otherwise, it returns the best solution that can be reached with the available memory.
- When enough memory is available for the entire search tree, the search is optimally efficient.

Summary

- **<u>Uninformed/Blind Search</u>**
  - ✔ DFS
  - ✔ BFS
  - ✔ Depth-limited search
  - ✔ IDS (or iterative deepening depth-first search)
  - ✔ Bidirectional
  - ✔ Uniform Cost
- **<u>Informed/Heuristic Search</u>**
  - ✔ Best-First Search
  - ✔ Greedy Best-First Search
  - ✔ A*
  - ✔ Memory Bounded Search
    - ✔ IDA*
    - ✔ RBFS
    - ✔ MA*
    - ✔ SMA*